Distributed Chat System

Real-time chat with distributed architecture: Node.js, Redis, NATS, Consul, Socket.IO.

Features

✅ Real-time messaging • Persistent sessions • Username uniqueness • Presence tracking • Message history • Horizontal scaling • Service discovery • Multi-room support • Auto-discovery • **Automatic dependency management**

Quick Start

Prerequisites

The script automatically checks for and offers to install missing dependencies:

- **podman** - Container runtime
- **curl** - HTTP client for API calls
- **python3** - For JSON parsing and cluster helpers
- **jq** - JSON processor
- **ip** (iproute2) - Network utilities

```
# Make scripts executable
chmod +x chat-system.sh setup-cluster.sh cluster_bridge.py
cluster_helper.py

# Run - dependencies will be checked automatically
./chat-system.sh start
```

PROF

If any dependencies are missing, you'll be prompted:

```
✗ Missing dependencies: podman curl jq
Would you like to install missing dependencies? [y/N]
```

Supported package managers: apt (Debian/Ubuntu), dnf (Fedora), yum (CentOS/RHEL), pacman (Arch), zypper (openSUSE)

Manual Installation (if needed)

```
# Debian/Ubuntu
sudo apt install podman curl python3 jq iproute2

# Fedora
sudo dnf install podman curl python3 jq iproute
```

```
# Arch Linux
sudo pacman -S podman curl python jq iproute2
```

Auto-Discovery (Recommended)

```
# Automatically discovers cluster and infrastructure
./chat-system.sh start --auto

# Start client
cd client-react && npm install && npm run dev
```

Access: http://localhost:5173

What happens:

- Checks if main cluster Consul is available (172.20.10.10:8500)
- Discovers existing Redis/NATS services
- Starts missing services as needed
- Registers with cluster for service discovery
- Falls back to standalone if no cluster found

Single Machine Standalone

```
./chat-system.sh start
cd client-react && npm install && npm run dev
```

Consul Cluster (Multi-Machine)

PROF

First node:

```
./chat-system.sh start --cluster --mode infrastructure
```

Additional nodes:

```
./chat-system.sh start --auto
```

Each Consul server runs Redis + NATS + Chat Node. NATS auto-clusters via Consul.

Setup (on each server, one at a time):

```
```

```
sudo ./setup-consul-chat-node.sh --auto-cluster
```

Firewall:

```
sudo ufw allow 3001/tcp 4222/tcp 6222/tcp 6379/tcp 8300:8302/tcp
8500/tcp 8600/tcp
```

Verify cluster:

```
# Check NATS cluster connections
curl http://localhost:8222/routez | jq '.routes'

# Check Consul service discovery
curl http://localhost:8500/v1/catalog/service/chat-nats | jq

# Test from another machine
curl http://FIRST_MACHINE_IP:8222/routez | jq '.routes'
```

Expected output:

- `routez` shows connected peer IPs in `routes` array
- Consul shows all registered `chat-nats` services
- Each node reports `num_routes` > 0 (except first node initially)

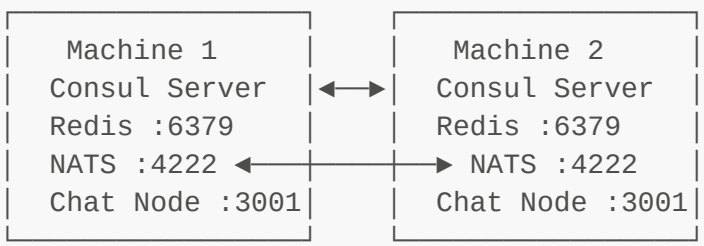
Architecture

Single Machine:

PROF

```
Client → Chat Nodes (3) → Redis + NATS + Consul
```

Distributed (each machine):



Messages sync across nodes via NATS cluster.

Testing Multi-Machine Setup

1. First Machine (Host)

```
# Check Consul is running
systemctl status consul

# Install chat stack
sudo ./setup-consul-chat-node.sh --auto-cluster

# Verify services
systemctl status chat-redis chat-nats chat-node

# Get your IP
ip addr show | grep 'inet ' | grep -v '127.0.0.1'

# Check NATS (should show no routes yet)
curl http://localhost:8222/routez | jq '.routes'
```

2. Second Machine (VM)

```
# Ensure Consul is clustered with first machine
consul members  # Should show both machines

# Install chat stack
sudo ./setup-consul-chat-node.sh --auto-cluster

# Verify services
systemctl status chat-redis chat-nats chat-node

# Check NATS cluster (should show connection to first machine)
curl http://localhost:8222/routez | jq
```

PROF

3. Verify Connection

On either machine:

```
# Check NATS cluster connections
curl http://localhost:8222/routez | jq '.routes'
# Expected: Array with peer connection info

# Check Consul service discovery
curl http://localhost:8500/v1/catalog/service/chat-nats | jq '.
[].Address'
# Expected: Both machine IPs listed

# View service logs
```

```
journalctl -u chat-node -f
# Expected: "Connected to NATS" message

# Test message sync (watch logs on both machines)
# Send message from one machine's client
# Should appear in logs on both machines
```

4. Test Chat Application

Machine 1:

```
cd client-react && npm install && npm run dev
# Open http://MACHINE1_IP:5173
# Login as "user1"
```

Machine 2:

```
cd client-react && npm install && npm run dev
# Open http://MACHINE2_IP:5173
# Login as "user2"
```

Expected behavior:

- user1 sees user2 join (presence event)
- Messages from user1 appear for user2 (and vice versa)
- Both see same message history

Success Indicators

PROF

✅ **NATS Cluster:** `routes` array not empty
✅ **Consul Registry:** Both IPs in service list
✅ **Message Sync:** Logs show same messages on both machines
✅ **Presence Sync:** Users see each other join/leave
✅ **No Errors:** `journalctl -u chat-node` shows no connection errors

Troubleshooting

NATS not clustering:

```
# Check firewall
sudo ufw status | grep 6222

# Test connectivity
nc -zv OTHER_MACHINE_IP 6222
```

```
# Restart NATS to retry connection
sudo systemctl restart chat-nats
sleep 5
curl http://localhost:8222/routez | jq '.routes'
```

Services not appearing in Consul:

```
# Check Consul connectivity
curl http://localhost:8500/v1/agent/self | jq '.Config.Datacenter'

# Re-register services
curl -X PUT http://localhost:8500/v1/agent/service/register -d
@/tmp/service.json
```

Messages not syncing:

```
# Check NATS connection in logs
journalctl -u chat-node -n 100 | grep -i nats

# Test NATS pub/sub manually
podman exec chat-nats nats pub test "hello"
# On other machine:
podman exec chat-nats nats sub test
```

Cluster Integration

How It Works

1. **Auto-Discovery**: Checks for main cluster Consul at `http://172.20.10.10:8500`
2. **Infrastructure Discovery**: Finds existing `redis-service` and `nats-service`
3. **Self-Registration**: Registers as `chat-service` with unique ID
4. **Peer Discovery**: Discovers other chat nodes automatically
5. **Unified Service**: External services see one `chat-service`, cluster handles load balancing

PROF

Service Discovery (Other Services)

```
from cluster_helper import pick_service_instance

# Get a chat node
address, port = pick_service_instance("chat-service")
response = requests.post(f"http://{address}:{port}/api/notify", json=
{...})
```

Autonomous Deployment

Each node autonomously:

- Discovers cluster infrastructure
- Starts missing services
- Registers itself
- Connects to peers
- **No manual IP configuration needed**

Management

Commands

```
./chat-system.sh start [--auto]        # Start with auto-discovery
(checks dependencies)
./chat-system.sh start                 # Standalone mode (checks
dependencies)
./chat-system.sh stop                  # Stop all services
./chat-system.sh restart               # Restart chat nodes
./chat-system.sh status                # Show status
./chat-system.sh logs <service>        # View logs (redis, nats, consul,
chat-1/2/3)
./chat-system.sh build                 # Rebuild Docker image (checks
dependencies)
./chat-system.sh clear-usernames       # Clear all registered usernames
from Redis
./chat-system.sh cluster-test [url]    # Test connectivity to cluster
Consul
./chat-system.sh deregister [url]      # Manually deregister services
from cluster
./chat-system.sh help                  # Show all options
```

Options

PROF

- `--auto` - Auto-discover cluster (recommended)
- `--cluster` - Force cluster mode
- `--cluster-consul <URL>` - Cluster Consul URL (default: http://172.20.10.10:8500)
- `--mode <mode>` - `standalone` | `infrastructure` | `node-only` | `auto`

Scaling

```
# Add a new node (automatically discovers and connects)
./chat-system.sh start --auto

# Remove a node (gracefully deregisters)
./chat-system.sh stop
```

Development

Backend

```
cd chat-node
npm install && npm run build
sudo podman build -t chat-node:latest .
```

Structure: `src/gateway.ts` (Socket.IO) • `src/chatcore.ts` (Redis→NATS) • `src/redis.ts` • `src/nats.ts` • `src/consul.ts`

Frontend

```
cd client-react
npm install && npm run dev
```

Structure: `src/pages/chatPage.tsx` • `src/components/` • `src/hooks/useSocket.tsx`

Technical Details

Message Flow

1. Client sends → Socket.IO (gateway.ts)
2. Write to local Redis Stream (persistence)
3. Publish to local NATS (real-time)
4. NATS cluster syncs to all nodes
5. All nodes receive → broadcast to clients

Data Storage

PROF

- **Redis Streams:** Message history (per room)
- **Redis Keys:** Username registry (case-insensitive, TTL 3600s)
- **NATS:** Ephemeral pub/sub (no storage)

Load Balancing

Client randomly selects node on connect. Production: use HAProxy/nginx.

Health Checks

- Chat nodes: `GET /health` → `{"status":"healthy","nodeId":"..."}`
- Consul polls every 10s

Configuration

Environment Variables (systemd services)

- `NODE_ID` - Hostname
- `PORT` - 3001 (default)
- `REDIS_URL` - redis://localhost:6379
- `NATS_URL` - nats://localhost:4222
- `CONSUL_URL` - http://localhost:8500

Client Config

Edit `client-react/src/hooks/useSocket.tsx` to change ports: `const ports = [3001, 3002, 3003];`

API Reference

Client → Server:

- `setUsername(string)` - Register
- `join({roomId})` - Join room
- `message({roomId, message})` - Send
- `typing({roomId, isTyping})` - Typing indicator

Server → Client:

- `usernameAccepted({username})` - Auth OK
- `joined({roomId})` - Room joined
- `history(Message[])` - Load history
- `message(Message)` - New message
- `userJoined/Left({username})` - Presence
- `userTyping({username})` - Typing

Project Structure

```
chat-system-test/
├── chat-system.sh                    # Single-machine manager
├── setup-consul-chat-node.sh         # Consul cluster installer
├── README.md
├── chat-node/                        # Backend (Node.js)
│   ├── src/{gateway,chatcore,redis,nats,consul}.ts
│   └── Dockerfile
└── client-react/                     # Frontend (React)
    └── src/{pages,components,hooks}/
```

Troubleshooting

Dependencies not installing automatically:

```
# Check your package manager
which apt-get dnf yum pacman zypper
```

```
# Manual install (Debian/Ubuntu)
sudo apt update && sudo apt install -y podman curl python3 jq iproute2

# Manual install (Fedora)
sudo dnf install -y podman curl python3 jq iproute
```

Nodes not discovering each other:

```
# Test cluster connectivity
curl http://172.20.10.10:8500/v1/agent/self

# Check if services are registered
curl http://172.20.10.10:8500/v1/catalog/service/chat-service

# View logs
./chat-system.sh logs chat-1
```

Services not starting:

```
# Rebuild image (also checks dependencies)
./chat-system.sh build

# Check Python dependencies
pip3 install requests

# Verify cluster helper scripts exist
ls -la cluster_bridge.py cluster_helper.py
```

Health checks failing:

PROF

```
# Test health endpoint
curl http://localhost:3001/health

# Check connectivity to infrastructure
redis-cli -h <redis-host> PING
curl http://<nats-host>:4222
```

Performance

- **Messages/sec:** ~1000/node
- **Concurrent users:** ~500/node
- **Latency:** <50ms (local network)

License

MIT